

nf_tables : dispersion architecturale — M-EPI-003

Transactions, générations et twins

Ted “Tedsig42” Kouhouenou ted.sig42@tutamail.com

Summary: Lire l’architecture transactionnelle de nf_tables, comprendre pourquoi trois CVE différents partagent la même racine structurelle, et apprendre à distinguer un twin réel d’une simple ressemblance de nom.

Prerequisites : M-EPI-000 à M-EPI-002, bases de C et de netfilter

Version: 1.0

Contents

I Instructions **2**

II Instructions Communes **5**

II.1 Structure du Dépôt 5

II.2 Qu'est-ce qu'une Preuve de Concept Valide 5

II.3 Processus de Soumission 6

II.4 Règle des Fichiers Supplémentaires Interdits 7

II.5 Format du Rapport 7

II.6 Cibles Autorisées Uniquement 8

II.7 Citez Toutes Vos Sources 8

III Instructions sur l'IA **10**

III.1 Aperçu de la Politique 10

III.2 Utilisations Autorisées 10

III.3 Utilisations Interdites 11

III.4 Responsabilité Personnelle 11

III.5 Citation Obligatoire de l'IA 11

III.6 Évaluation par les Pairs et l'IA 12

III.7 Rappel Éthique 13

IV Avant-propos **14**

V Aperçu du module **15**

VI Exercise 00 **16**

VII Exercise 01 **17**

VIII Exercise 02 **19**

IX Exercise 03 **21**

X Exercise 04 **23**

XI Rendu et évaluation **24**

XII Ressources par exercice **25**

XIII Sources **27**

Chapter I

Instructions

- Une journée. Le module est dense : lisez les sections avant de coder.
- Travaillez sur une copie locale du noyau, jamais sur une machine de production.
- Aucun exploit, aucun module noyau chargeable sur une cible réelle.
- Les schémas et les commandes font partie du rendu.

Format du rendu — en-tête obligatoire

Tout fichier rendu dans ce module doit commencer par l'en-tête ci-dessous, **adapté à la syntaxe de commentaire du langage** utilisé.

Modèle générique

```
/**
 * Module: <code module>
 * Exercise: <numéro exo>
 * File: <nom du fichier>
 * Author: <votre login JeuneCrack>
 *
 * Description: <une phrase, pas de roman>
 *
 * Return: <voir ci-dessous>
 */
```

Login

Votre identifiant **JeuneCrack** (ex: `tedsig42`). Pour un rendu en binôme : `login1/login2`. Ajoutez un handle Matrix après le login si vous le souhaitez (ex: `tedsig42 @tedsig42`).

Return — selon le type de fichier

Type de fichier	Valeur de Return
C (`.c`)	`return (0);` (succès), `return (-1);` (erreur), `return (valeur);` (résultat calculé)
Python (`.py`)	`0` (succès), `1` (erreur), `N/A` (analyse, script sans exit)
Shell (`.sh`)	`exit 0` (succès), `exit 1` (erreur)
Markdown (`.md`)	`N/A` — les fichiers d'analyse n'ont pas de valeur de retour
Exploit C/Python	`shell` (shell obtenu), `flag: <flag>` (flag capturé), `proof: <hash>` (preuve de concept)
Binaire (`.bin`)	`N/A` — pas d'en-tête pour les fichiers binaires

Exemples par langage

C (`.c`) :

```c
/**
 * Module: F-EXP-001
 * Exercise: ex02
 * File: exploit.c
 * Author: tedsig42
 *
 * Description: Buffer overflow with ret2libc
 *
 * Return: 0
 */
```

Python (`.py`) :

```python
# Module: F-WEB-001
# Exercise: ex05
# File: ssrf-exploit.py
# Author: tedsig42
#
# Description: Blind SSRF bypass via redirect
#
# Return: 0
```

Shell (`.sh`) :

```shell
# Module: F-SUP-001
# Exercise: ex03
# File: dependency-confusion.sh
# Author: tedsig42
#
# Description: Automated dependency confusion test
#
# Return: exit 0
```

Markdown (`.md`) — analyse, rapport :

```
<!--
Module: M-SRC-001
Exercise: ex04
File: audit-report.md
Author: tedsig42

Description: Code audit of vulnerable-httpd

Return: N/A
-->
```

Chapter II

Instructions Communes

II.1 Structure du Dépôt

Votre dépôt Git doit respecter la convention de nommage suivante pour tous les modules soumis. L'outil d'évaluation (`pendragon`) se base sur cette structure pour localiser et tester votre travail.

```
<login>/
+-- <MODULE-ID>-<slug>/         # ex. M-BLS-001-business-logic/
|   +-- <turnindir>/            # ex. ex00-pricing-manipulation/
|   |   +-- <file1>             # ex. poc-pricing.py
|   |   +-- <file2>             # ex. report.md
|   |   +-- ...
|   +-- <turnindir>/            # ex. ex01-race-conditions/
|   |   +-- ...
|   +-- ...
+-- ...
```

- **Répertoire du module** : le nom complet du répertoire tel qu'affiché dans votre curriculum (ex. `M-BLS-001-business-logic`).
- **Répertoire de rendu** : la valeur de `\turnindir` fournie dans chaque exercice (ex. `ex00-pricing-manipulation`).
- **Fichiers de l'exercice** : exactement les fichiers listés dans `\exfiles`. Ni fichiers supplémentaires, ni fichiers manquants.

II.2 Qu'est-ce qu'une Preuve de Concept Valide

Une PoC doit être :

1. **Reproductible**. N'importe qui avec le même environnement peut l'exécuter et observer la vulnérabilité. Les chemins codés en dur, les adresses IP absolues et les jetons expirés ne sont pas acceptables.

2. **Documentée**. Chaque étape est expliquée. L'évaluteur ne devrait pas avoir à deviner ce que fait le script.

3. **Ciblée**. L'exercice spécifie une cible autorisée (lab, maquette, binaire local ou plateforme désignée). Ne testez jamais les PoC sur des systèmes de production qui ne vous appartiennent pas.

4. **Minimale**. La PoC démontre *uniquement* la vulnérabilité en question. Les charges utiles non liées et le bruit sont pénalisés.

① Une PoC qui modifie silencieusement l'état du système, supprime des fichiers ou persiste des changements en dehors du périmètre de l'exercice sera rejetée. Si votre exploit nécessite un nettoyage, documentez les étapes de nettoyage dans votre rapport.

II.3 Processus de Soumission

1. Réalisez les livrables de l'exercice sur votre machine locale.

2. Committez et poussez vers votre dépôt Git assigné.

3. Le dépôt est gelé après la date limite.

4. Les pairs clonent votre dépôt et évaluent en utilisant :

   - `pendragon` pour l'exécution automatisée des tests (là où le module fournit des configurations mouli).
   - La grille manuelle pour la qualité du rapport, le style de code et la méthodologie.



Exécutez toujours `pendragon run <MODULE-ID> <ex_id> <path/to/student/repo>` avant votre commit final pour vérifier que votre soumission passe les vérifications automatisées.

Test avec pendragon

Si le module inclut une configuration mouli (fichiers JSON dans `resources/mouli/`), vous pouvez tester votre soumission localement :

```
pendragon run M-BLS-001 ex00 ~/student-repos/tedsig42
```

`pendragon` résout le chemin de l'exercice automatiquement en cherchant `<mod-fullname>/<ex-dirname>/` dans le répertoire étudiant donné.

Vérification de style avec caliban

Pour les modules qui nécessitent du code C, `caliban` vérifie la conformité du style :

```
caliban check ~/student-repos/tedsig42/M-BLS-001-business-logic
```

`caliban` applique la Norme 42 (taille des fonctions, nommage des variables, constructions interdites) adaptée aux contextes de recherche de vulnérabilités. Exécutez-le avant la soumission — un échec aux vérifications de style peut invalider l'exercice.

II.4 Règle des Fichiers Supplémentaires Interdits

- Votre dépôt doit contenir **uniquement** les fichiers listés dans `\exfiles` pour chaque exercice.
- Fichiers interdits : `.DS_Store`, `__pycache__`, `.venv/`, `node_modules/`, `.git/`, binaires compilés (sauf explicitement listés dans `exfiles`), fichiers d'échange d'éditeur, fichiers journaux et tout fichier non demandé.
- Utilisez `.gitignore` pour exclure automatiquement les fichiers indésirables.

II.5 Format du Rapport

Lorsqu'un rapport (`report.md`) est requis, il doit inclure :

1. **Classe de vulnérabilité** (identifiant CWE si applicable).
2. **Étapes de reproduction** — numérotées, prêtes à être copiées-collées.
3. **Charges utiles** — chaînes exactes, requêtes ou entrées utilisées.
4. **Impact** — ce qu'un attaquant peut accomplir.
5. **Mitigation** — comment le développeur devrait corriger.
6. **Sources** — toutes les ressources consultées (voir Section II.7).

En-tête de Rendu

Chaque fichier source doit inclure l'en-tête de rendu défini dans `../../../resources/latex/deliverable-header.tex`, adapté à la syntaxe de commentaire du langage utilisé.

II.6 Cibles Autorisées Uniquement

① Vous pouvez **uniquement** tester les cibles explicitement listées dans l'exercice (labs, serveurs locaux, maquettes, binaires désignés). Tester des systèmes de production réels, des applications tierces ou toute cible qui ne vous appartient pas ou pour laquelle vous n'avez pas d'autorisation écrite est interdit et peut entraîner une disqualification immédiate.

L'éthique du bug bounty s'applique tout au long de ce curriculum :

- Aucun test sur des programmes en production sans un accord de périmètre valide.
- Aucun déni de service sauf si l'exercice l'exige explicitement.
- Aucune exfiltration de données au-delà de ce qui est nécessaire pour démontrer la vulnérabilité.
- Si vous accédez accidentellement à des données réelles d'utilisateurs, arrêtez immédiatement et documentez ce qui s'est passé.

II.7 Citez Toutes Vos Sources

Toute ressource consultée pendant votre travail doit être citée. Cela inclut :

- Articles académiques et billets de blog.
- Rapports publics de bug bounty (HackerOne, Bugcrowd, etc.).
- Documentation officielle (RFC, pages de manuel, documentation fournisseur).
- **Outils d'IA** : nom du modèle, résumé de la question, date et méthode de vérification (voir le chapitre Instructions IA pour le format de citation exact).
- Extraits de code adaptés de projets open-source (avec attribution de licence si requis).

Format de citation

```
# R\'ef\'erences

- [1] PortSwigger, "Business Logic Flaws",
  https://portswigger.net/web-security/logic-flaws
  (consult\'e le 2025-03-15).
- [2] Rapport HackerOne #123456, "IDOR on /api/users",
  anonymis\'e, consult\'e via divulgation publique.
- [3] G\'en\'er\'e avec l'assistance de Claude 3.5 Sonnet
  le 2025-03-14. Question : "Write a Python script that...".
  V\'erifi\'e par : revue manuelle de chaque requ\^ete HTTP.
```

Attribution en ligne

Si un exercice vous demande explicitement d'utiliser une technique ou un outil spécifique, vous n'avez pas besoin de citer l'exercice lui-même comme source. Citez tout ce qui se trouve en dehors de l'énoncé de l'exercice.

Chapter III

Instructions sur l'IA

III.1 Aperçu de la Politique

Les outils de codage assistés par IA (ChatGPT, Claude, Copilot, Gemini, Cursor, Codeium, et similaires) sont **autorisés** tout au long de ce cursus. La recherche en sécurité est un domaine où la maîtrise de l'IA devient essentielle — apprendre à utiliser ces outils efficacement fait partie de votre formation.

Cependant, l'objectif est la **compréhension**, pas l'automatisation. Si vous ne pouvez pas expliquer chaque ligne de votre exploit sans l'aide de l'IA, vous n'avez pas terminé l'exercice.

① Utiliser l'IA pour contourner l'objectif pédagogique d'un exercice — par exemple, lui demander de rétro-ingéniérer un binaire pour ne pas avoir à apprendre le désassembleur, ou de générer une chaîne d'exploitation complète que vous ne comprenez pas — va à l'encontre du but du cursus et sera traité comme une fraude académique.

III.2 Utilisations Autorisées

- **Explication de concepts**. Demander à l'IA de clarifier une classe de vulnérabilité, un sous-système du noyau, ou un protocole réseau.
- **Échafaudage de code**. Générer du code de départ pour un harnais, un fuzzer, ou un squelette de PoC — puis le modifier et l'étendre vous-même.
- **Débogage**. Expliquer des messages d'erreur, la sortie d'un désassembleur, ou un comportement de crash inattendu.
- **Résumé de documentation**. Condenser une longue RFC ou un article technique en notes exploitables.
- **Traduction**. Traduire du contenu technique entre l'anglais et le français (les deux langues sont utilisées dans ce cursus).

- **Rédaction de rapports**. Demander de l'aide pour structurer un rapport de bug bounty — mais remplir le contenu technique vous-même.

III.3 Utilisations Interdites

- **Génération complète d'exploit**. Demander à l'IA d'écrire l'exploit complet d'un exercice, puis le soumettre sans le comprendre.
- **Contournement des contraintes de l'exercice**. Utiliser l'IA pour contourner une restriction que l'exercice impose explicitement (p. ex. « n'utilisez pas de scanners automatisés »).
- **Découverte automatisée de vulnérabilités**. Exécuter des outils de scan pilotés par IA qui remplacent l'analyse manuelle que vous êtes censé pratiquer.
- **Rédaction des grilles d'évaluation**. Vous n'êtes pas autorisé à utiliser l'IA pour générer des critères d'évaluation par les pairs ou des réponses.
- **Plagiat sans attribution**. Copier le résultat de l'IA sans le citer (voir Section III.5).

III.4 Responsabilité Personnelle

Lorsque vous demandez à l'IA de générer du code d'exploit, vous devenez responsable de chaque octet de ce résultat avant de l'utiliser contre une cible.

1. **Lisez le code généré** ligne par ligne.
2. **Identifiez toutes les hypothèses** faites par l'IA (architecture, version du noyau, drapeaux du compilateur, topologie réseau).
3. **Testez en isolation** avant d'exécuter contre une cible d'exercice.
4. **Comprenez les modes d'échec**. Si l'exploit généré plante ou se bloque, vous devez diagnostiquer et corriger le problème vous-même.



Si l'IA génère du code que vous ne comprenez pas, demandez-lui de l'expliquer. Si vous ne comprenez toujours pas après l'explication, le code n'est pas prêt à être utilisé. Passez à une approche plus simple que vous maîtrisez entièrement.

III.5 Citation Obligatoire de l'IA

Chaque fichier contenant du code généré ou assisté par l'IA doit inclure un commentaire de citation. Le format est :

```
# Generated with assistance from <tool> on <date>.
# Prompt: <one-line summary of the prompt>
# Verified by: <method used to verify the output>
```

Exemples

Python :

```
# Generated with assistance from Claude 3.5 Sonnet on 2025-03-14.
# Prompt: "Write a Python script that sends HTTP requests with
# modified price fields to /api/checkout/pay"
# Verified by: manual review of each request and response
```

C :

```
/* Generated with assistance from GitHub Copilot on 2025-03-14.
 * Prompt: "Implement a stack-based buffer overflow with
 * ret2libc for x86_64 Linux"
 * Verified by: compiled with -fno-stack-protector and
 * tested against local binary with ASLR disabled */
```

Markdown (rapport) :

```
<!--
Generated with assistance from ChatGPT-4 on 2025-03-14.
Prompt: "Structure a bug bounty report for IDOR with
sections: summary, steps, impact, mitigation"
Verified by: all technical content written manually,
AI only suggested structure
-->
```

Interactions multiples avec l'IA

Si vous avez utilisé plusieurs outils d'IA, ou itéré à travers plusieurs invites, citez chaque interaction significative. Un groupe d'invites connexes peut être résumé :

```
<!--
AI assistance used in this file:
- Claude 3.5 Sonnet (2025-03-14): harness skeleton generation
- ChatGPT-4 (2025-03-14): crash triage help
- Copilot (2025-03-15): inline autocomplete for loop logic
All generated code reviewed and modified manually.
-->
```

III.6 Évaluation par les Pairs et l'IA

Lors de l'évaluation par les pairs, l'évaluateur peut :

- Vous demander d'**expliquer tout code généré par l'IA** ligne par ligne.

- Vous demander de **reproduire le raisonnement** sans l'aide de l'IA.
- Demander à voir votre **historique d'invites** pour un exercice donné.
- Déduire des points si la citation de l'IA est manquante ou si vous ne pouvez pas expliquer le code que l'IA a généré.

① Présenter le code généré par l'IA comme votre propre travail (défaut de citation) est du plagiat. Si l'évaluateur soupçonne du plagiat, il peut saisir l'équipe pédagogique pour examen.

III.7 Rappel Éthique

- Les modèles d'IA peuvent produire du code incorrect, dangereux, ou illégal. Vous êtes **responsable** de vérifier chaque résultat avant exécution.
- Les modèles d'IA entraînés sur la recherche publique en sécurité peuvent reproduire textuellement du code provenant de projets sous licence GPL. Vérifiez les licences avant d'intégrer ces éléments dans votre travail.
- Ne saisissez pas de code sensible ou propriétaire dans des outils d'IA hébergés dans le cloud. Utilisez des modèles locaux (Llama, Mistral, DeepSeek) pour les travaux impliquant des cibles non publiques.
- L'éthique de la recherche en sécurité que vous apprenez ici s'applique à l'utilisation de l'IA : ne demandez pas à l'IA de générer des malwares, des rançongiciels, ou du code destiné à nuire à des systèmes en dehors du cadre de l'exercice.



La maîtrise des outils d'IA est une compétence recherchée dans la chasse aux bogues moderne. Utilisez-les sagement, citez-les honnêtement, et conservez toujours votre propre compréhension de chaque exploit que vous déployez.

Chapter IV

Avant-propos

nf_tables est un *Logical Engine* au sens du papier : des objets (règles, sets, éléments, chaînes) sont créés, activés, désactivés et détruits dans des transactions. Le sous-système utilise deux idées centrales :

- **la génération** : une transaction prépare un état futur sans modifier immédiatement l'état visible ;
- **le commit** : le passage d'une génération à la suivante rend les changements visibles, tandis que les anciens objets peuvent encore être référencés par des lecteurs concurrents.

Quand un correctif ajoute un test dans une fonction, il faut donc demander : *ce test porte-t-il sur la génération courante ou sur la génération suivante ?* C'est exactement la question qui distingue `nft_is_active()` de `nft_is_active_next()`.



L

e papier parle de *architectural scatter* : la même forme de bug apparaît dans plusieurs sous-systèmes parce que la racine est architecturale, pas parce qu'un correctif a été copié. nf_tables en est l'exemple le plus lisible.

Chapter V

Aperçu du module

Ex	Titre	Difficulté	Points	
00	Carte d'architecture	Facile	3	
01	CVE-2023-35001 : union et stride	Moyen	4	
02	CVE-2024-1086 : NF_DROP et errno	Moyen	4	
03	CVE-2023-4244 vers CVE-2024-1085	Difficile	5	
04	Checklist de candidat	Difficile	4	
	Total		**20**	

Chapter VI

Exercise 00

	Exercise 00
Carte d'architecture	
Turn-in directory : `ex00/`	
Files to turn in : `architecture.md`	
Allowed functions : `grep, ctags, un éditeur de texte`	

Dans `architecture.md`, dessinez ou décrivez la relation entre :

1. une transaction (`struct nft_trans`) ;
2. une génération (`genmask`, `nft_genmask`) ;
3. un set et un élément de set ;
4. les fonctions `nft_is_active()` et `nft_is_active_next()` ;
5. le chemin de commit et le chemin de nettoyage (GC, abort, destroy).

Ajoutez une légende qui répond à ces questions :

- Qui voit quel état pendant une transaction ?
- Pourquoi un élément peut-il être actif pour un lecteur et inactif pour un autre ?
- Quels objets peuvent survivre au commit et être libérés plus tard ?



Ne recopiez pas la documentation mot pour mot. Votre schéma doit montrer *qui lit quoi, quand.*

Chapter VII

Exercise 01

	Exercise 01
CVE-2023-35001 : union et stride	
Turn-in directory : `ex01/`	
Files to turn in : `cve-2023-35001.md`	
Allowed functions : `git, grep, un éditeur de texte`	

CVE-2023-35001 est un accès hors limites dans l'évaluation de `nft_byteorder`. Le papier le classe comme un cas de dispersion architecturale, pas comme un simple bug de validation d'entrée.

Mission

Dans `cve-2023-35001.md` :

1. Décrivez la structure `nft_byteorder` et la manière dont les champs de registre sont lus.

2. Expliquez pourquoi une valeur d'entrée apparemment valide peut produire un accès hors limites.

3. Quelle hypothèse le correctif ajoute-t-il ? Porte-t-il sur la valeur, sur l'index, ou sur la taille ?

4. Pourquoi ce correctif ne suffit-il pas à protéger les autres opérations qui manipulent des registres ?

Sources

- NVD : https://nvd.nist.gov/vuln/detail/CVE-2023-35001

- Analyse publique : https://pwning.tech/nftables/
- Commit de correction : https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=c1592a89942e9678f7d9c8030efa777c0d57edab

Chapter VIII

Exercise 02

	Exercise 02
CVE-2024-1086 : NF_DROP et errno	
Turn-in directory : `ex02/`	
Files to turn in : `cve-2024-1086.md`	
Allowed functions : `git, grep, un éditeur de texte`	

CVE-2024-1086 est un double free dans le chemin de verdict de nf_tables. Le point important n'est pas la valeur de retour elle-même, mais ce que le reste de la pile fait de cette valeur.

Mission

Dans `cve-2024-1086.md` :

1. Décrivez le chemin `nft_verdict_init()` vers `nf_hook_slow()`.
2. Pourquoi un verdict `NF_DROP` accompagné d'un errno positif est-il dangereux ?
3. Quelle hypothèse le correctif encode-t-il sur les valeurs de retour autorisées ?
4. Cherchez un autre appelant qui suppose la même chose sans le vérifier. Donnez le nom de la fonction et la commande utilisée.



L

a correction de CVE-2024-1086 ne rend pas nf_tables « sûr ». Elle ferme un chemin précis. Votre analyse doit dire lequel, et lequel reste ouvert.

Chapter IX

Exercise 03

	Exercise 03
CVE-2023-4244 vers CVE-2024-1085	
Turn-in directory : `ex03/`	
Files to turn in : `mutation-chain.md`	
Allowed functions : `git, grep, un éditeur de texte`	

Deux correctifs, cinq mois d'écart, le même sous-système. Le premier retire la marque « busy » et l'API de batch GC. Le second remplace un test de génération dans le chemin de désactivation des éléments catchall.

```
- if (!nft_is_active(net, ext))
+ if (!nft_is_active_next(net, ext))
      continue;
```

Mission

Dans `mutation-chain.md` :

1. Identifiez le commit du premier correctif et ce qu'il retire exactement.

2. Identifiez le commit du second correctif et la fonction modifiée.

3. Expliquez pourquoi le second bug était déjà présent avant le premier correctif, mais *atteignable* après.

4. Classez la relation : *incomplete-fix variant* ou *architectural scatter*. Justifiez avec la définition du papier.

5. Formulez la question d'audit qui aurait permis de trouver le second bug en lisant le premier diff.

6. Comparez avec le double-free pipapo de CVE-2024-26809 : quel drapeau (`priv->dirty`, `set->dead`) rend le chemin de destruction atteignable, et pourquoi le correctif de CVE-2024-1085 ne le ferme pas ?

Sources

- Commit `a2dd0233cbc4` : suppression de la marque busy et de l'API de batch GC.
- Commit `b1db244ffd041a49ecc9618e8feb6b5c1afcdaa7` : correction de CVE-2024-1085.
- NVD : https://nvd.nist.gov/vuln/detail/CVE-2024-1085

Chapter X

Exercise 04

	Exercise 04
Checklist de candidat	
Turn-in directory : `ex04/`	
Files to turn in : `candidate.md queries/`	
Allowed functions : `grep, ctags, un éditeur de texte`	

Choisissez une fonction de nf_tables qui manipule un élément de set ou un verdict. Construisez une fiche de candidat, pas un exploit.

La fiche doit contenir :

1. **Objet** : quel objet est créé, activé, désactivé ou libéré ?
2. **Chemin nominal** : la séquence d'appels heureuse.
3. **Chemins d'abandon** : abort, GC, commit partiel, erreur.
4. **Génération** : le code lit-il l'état courant ou l'état suivant ?
5. **Twins** : trois fonctions voisines et leur statut.
6. **Requêtes** : les commandes `grep`/`ctags` utilisées, stockées dans `queries/`.
7. **Verdict** : faux positif, candidat à investiguer, ou candidat à prioriser, avec un score de triage.

Chapter XI

Rendu et évaluation

```
M-EPI-003-nf-tables/
|-- ex00/architecture.md
|-- ex01/cve-2023-35001.md
|-- ex02/cve-2024-1086.md
|-- ex03/mutation-chain.md
+-- ex04/
    |-- candidate.md
    +-- queries/
```

Critère	**Attendu**	**Points**
Architecture	Génération et commit expliqués sans confusion	4
CVE-2023-35001	Lien entre structure, index et OOB	4
CVE-2024-1086	Chemin de verdict et double free	4
Mutation	Distinction correcte entre les deux modes	4
Candidat	Fiche reproductible, requêtes fournies	4

Chapter XII

Ressources par exercice

Exercice 00 — Carte d'architecture

- nf_tables : https://www.kernel.org/doc/html/latest/networking/nf_tables.html
- Papier, section 6.1.
- Sources noyau : https://git.kernel.org/
- À chercher : `nft_trans`, `genmask`, `nft_setelem`, `commit_list`.

Exercice 01 — CVE-2023-35001

- NVD : https://nvd.nist.gov/vuln/detail/CVE-2023-35001
- Analyse : https://pwning.tech/nftables/
- Papier, section 6.2.
- À chercher : `nft_byteorder`, `union`, `priv->len`, `stride`.

Exercice 02 — CVE-2024-1086

- NVD : https://nvd.nist.gov/vuln/detail/CVE-2024-1086
- Papier, section 6.2.
- nf_tables : https://www.kernel.org/doc/html/latest/networking/nf_tables.html
- À chercher : `NF_DROP`, `NF_ACCEPT`, `verdict`, `double free`, `nf_hook_slow`.

Exercice 03 — CVE-2023-4244 vers CVE-2024-1085

- NVD, CVE-2023-4244 : https://nvd.nist.gov/vuln/detail/CVE-2023-4244
- NVD, CVE-2024-1085 : https://nvd.nist.gov/vuln/detail/CVE-2024-1085
- NVD, CVE-2024-26809 : https://nvd.nist.gov/vuln/detail/CVE-2024-26809

- OtterSec : https://osec.io/blog/netfilter-universal-root-1-day/
- Commit `b1db244ffd041a49ecc9618e8feb6b5c1afcdaa7`.
- À chercher : `nft_setelem_catchall_deactivate`, `set->dead`, `priv->dirty`, `nft_pipapo_destroy`.

Exercice 04 — Checklist de candidat

- Papier, section 18.8.
- Dépôt compagnon : `rules/TRIAGE.md`.
- Semgrep : https://semgrep.dev/docs/
- Weggli : https://github.com/weggli-rs/weggli
- À chercher : `nft_is_active`, `_deactivate`, `_destroy`, `genmask`.

Chapter XIII

Sources

- Phrack, *Security is failure: Every Patch Is a Guess*, sections 6.2, 6.3 et 14.
- Documentation nf_tables : https://www.kernel.org/doc/html/latest/networking/nf_tables.html
- `pwning.tech/nftables` : analyse de CVE-2023-35001.
- OtterSec, *OtterRoot: Netfilter universal root 1-day* : https://osec.io/blog/netfilter-universal-root-1-day/
- NVD : CVE-2023-35001, CVE-2024-1086, CVE-2023-4244, CVE-2024-1085, CVE-2024-26809.